

AW87XXX Android Driver(SPRD)

版本： V3.4

时间： 2022 年 11 月

目录

1. 驱动说明	3
2. 驱动移植	3
2.1 AW87XXX 驱动移植	3
2.1.1 DTS 配置	3
2.1.2 驱动配置	4
2.1.3 BIN 文件配置	5
2.2 平台驱动配置	6
2.2.1 KCONTROL 注册	6
2.2.2 PA 场景配置	6
2.3 驱动移植有效性验证	8
3. 调试接口	9
3.1 设备节点	9
4. 附录	10
4.1 常见问题	10
4.1.1 BIN 文件加载失败	10

1. 驱动说明

驱动源码文件	aw87xxx.c, aw87xxx.h, aw87xxx_device.c, aw87xxx_device.h, aw87xxx_monitor.c, aw87xxx_monitor.h, aw87xxx_dsp.c, aw87xxx_dsp.h, aw87xxx_acf_bin.c, aw87xxx_acf_bin.h, aw87xxx_log.h, aw87xxx_bin_parse.c, aw87xxx_pid_39_reg.h, aw87xxx_pid_59_3x9_reg.h, aw87xxx_pid_59_5x9_reg.h, aw87xxx_pid_5a_reg.h, aw87xxx_pid_18_reg.h, aw87xxx_pid_9b_reg.h, aw87xxx_pid_76_reg.h, aw87xxx_pid_60_reg.h
驱动支持产品	aw87319, aw87329, aw87339, aw87349, aw87359, aw87389, aw87509, aw87519, aw87529, aw87539, aw87549, aw87559, aw87569, aw87579, aw81509, aw87579G, aw87390, aw87360, aw87401, aw87390G aw87358, aw87560, aw87550,
I ² C 地址范围	aw87358:0x5C 其他产品:0x58, 0x59, 0x5A, 0x5B

2. 驱动移植

2.1 AW87XXX 驱动移植

2.1.1 DTS 配置

AW87359/AW87389/AW87549/AW87390/AW87360/AW87401/AW87390G 产品没有复位引脚，不能配置 reset-gpio。

产品表信息表

产品	产品 ChipID	是否有 reset-pin
AW87319	0x9B	是
AW87329/AW87339/AW87349	0x39	是
AW87519/AW87529	0x59	是
AW87359/AW87389	0x59	否
AW87549	0x5A	否
AW87559/AW87569/AW87579/AW81509/AW87579G	0x5A	是
AW87390/AW87360/AW87401/AW87390G	0x76	否
AW87358	0x18	是
AW87560/AW87550	0x60	是

单 PA 配置方法

&i2c_x {	/*x 表示 I2C 总线*/
----------	-----------------

```
aw87xxx_pa@58 {                                     /*以 I2C 地址 0x58 举例*/
    compatible = "awinic,aw87xxx_pa";
    reg = <0x58>;
    /* AW87359/AW87389/AW87549/AW87390/AW87360/AW87401/AW87390G 不能配置 reset-
gpio, 否则无法正常加载配置*/
    reset-gpio = <&ap_gpio 1 0>; /*reset-gpio 以 gpio 1 举例*/
    dev_index = < 0 >;          /*单 PA dev_index 为 0*/
    status = "okay";
};
};
```

多 PA 配置方法

```
&i2c_x {                                             /*x 表示 i2c 的总线号*/
    aw87xxx_pa@58 {
        compatible = "awinic,aw87xxx_pa";
        reg = <0x58>;
        /* AW87359/AW87389/AW87549/AW87390/AW87360/AW87401/AW87390G 不能配置 reset-
gpio, 否则无法正常加载配置*/
        reset-gpio = <&ap_gpio 1 0>;
        dev_index = < 0 >;          /*第 1 个 PA dev_index 为 0*/
        status = "okay";
    };
    aw87xxx_pa@59 {
        compatible = "awinic,aw87xxx_pa";
        reg = <0x59>;
        /* AW87359/AW87389/AW87549/AW87390/AW87360/AW87401/AW87390G 不能配置 reset-
gpio, 否则无法正常加载配置*/
        reset-gpio = <&ap_gpio 7 0>;
        dev_index = < 1 >;          /*第 2 个 PA dev_index 为 1*/
        status = "okay";
    };
};
/*以上以双 PA 举例, 多 PA 时 dev_index 依次增加, 其他属性参考单 PA 配置*/
```

2.1.2 驱动配置

defconfig 编译配置选项:

```
CONFIG_SND_SOC_AW87XXX=y
```

内核 codecs 目录下创建 aw87xxx 目录, 添加驱动文件:

```
aw87xxx.c, aw87xxx_device.c, aw87xxx_monitor.c, aw87xxx_dsp.c, aw87xxx_acf_bin.c,
aw87xxx_bin_parse.c aw87xxx.h, aw87xxx_device.h, aw87xxx_monitor.h, aw87xxx_dsp.h,
aw87xxx_acf_bin.h, aw87xxx_log.h, aw87xxx_pid_39_reg.h, aw87xxx_pid_59_3x9_reg.h,
aw87xxx_pid_59_5x9_reg.h, aw87xxx_pid_5a_reg.h, aw87xxx_pid_18_reg.h,
aw87xxx_pid_9b_reg.h, aw87xxx_pid_76_reg.h, aw87xxx_pid_60_reg.h
```

Kconfig 配置:

```
config SND_SOC_AW87XXX
    tristate "SoC Audio for awinic AW87XXX Smart K PA"
    depends on I2C
help
    This option enables support for AW87XXX Smart K PA.
```

Makefile 配置:

```
obj-$(CONFIG_SND_SOC_AW87XXX) += aw87xxx/aw87xxx.o
aw87xxx/aw87xxx_device.o aw87xxx/aw87xxx_monitor.o
aw87xxx/aw87xxx_bin_parse.o aw87xxx/aw87xxx_dsp.o
aw87xxx/aw87xxx_acf_bin.o
```

2.1.3 bin 文件配置

PA 需要配置寄存器等参数才能正常工作，PA bin 文件配置步骤如下:

编译配置

在项目对应位置添加 bin 文件编译选项:

```
PRODUCT_COPY_FILES += \
    xxx/aw87xxx_acf.bin:$(TARGET_COPY_OUT_VENDOR)/firmware/aw87xxx_acf.bin
/*xxx 为平台路径*/
```

路径配置

在内核 firmware_class.c 文件中添加 bin 文件在手机中的目录，一般目录为/vendor/firmware

```
static const char * const fw_path[] = {
    fw_path_para,
    "/vendor/firmware",
    "/lib/firmware/updates/" UTS_RELEASE,
    "/lib/firmware/updates",
    "/lib/firmware/" UTS_RELEASE,
    "/lib/firmware"
};
```

/* 注: 调试阶段可直接 push bin 文件到 /vendor/firmware/ */

bin 文件选择

以 aw87390 为例，bin 文件包含 Music/Receiver/Off 场景:

```

aw87xxx_driver                                /*驱动根目录*/
├── config                                    /*PA配置路径*/
│   ├── aw87390
│   │   ├── mono
│   │   │   └── aw87xxx_acf.bin                /*单PA 配置*/
│   │   └── stereo
│   │       └── aw87xxx_acf.bin                /*双PA 配置*/
│   └── monitor_bin_guide                    /*monitor bin目录*/
│       ├── aw87xxx_monitor.bin
│       ├── aw87xxx_monitor_bin_guide.pdf
│       ├── monitor.exe
│       └── monitor_params.txt

```

2.2 平台驱动配置

2.2.1 kcontrol 注册

aw87xxx_codec 注册:

```

/*以 UMS512 平台为例, 修改 sprd-codec.c */
@@ -195,6 +195,8 @@ static int sprd_codec_get_ctrl(struct snd_soc_codec *codec,
char *name);
static void sprd_codec_psg_state_init(struct snd_soc_codec *codec);
static void sprd_codec_psg_state_exit(struct snd_soc_codec *codec);
/*添加驱动接口*/
+ extern int aw87xxx_add_codec_controls(void *codec);
+
static unsigned long sprd_codec_dp_base;

enum {
@@ -3955,6 +3957,13 @@ static int sprd_codec_soc_probe(struct snd_soc_codec
*codec)
    return ret;
}

/*添加注册调用*/
+ ret = aw87xxx_add_codec_controls((void*) codec);
+ if(ret < 0) {
+     pr_err("add control failed\n");
+ }
+
return 0;
}

```

2.2.2 PA 场景配置

配置 DTS:

```

/*以 UMS512 平台为例, 修改 ums512-mach.dtsi */
& sound_vbc_v4_sprd_codec {
    status = "okay";
}

```

```
+ /*<0 0 4 0>分别表示 ext_control_type, hook selection, mode, gpio share*/
+ sprd,spk-ext-pa-info = <0 0 4 0>;
+ sprd,spk-ext-pa-gpio = <&ap_gpio 156 0>;
};
/*hook selection 代表下面 ext_hook_arr 数组 aw87xxx 接口的位置*/
```

添加设置场景接口:

```
/*以 UMS512 平台为例, 修改 sprd-asoc-card-utils-hook.c */
@@ -106,6 +106,32 @@ static int ext_debug_sysfs_init(void)
    return ret;
}

#ifdef CONFIG_SND_SOC_AW87XXX
extern int aw87xxx_set_profile(int dev_index, char *profile);
+
+static char *aw_profile[] = {"Music", "Off"}; /*aw87xxx_acf.bin 文件配置场景*/
+enum aw87xxx_dev_index {
+    AW_DEV_0 = 0,
+};
+
+static int hook_spk_aw87xxx(int id, int on)
+{
+    pr_info("%s id: %d, on: %d\n", __func__, id, on);
+    if(on)
+        aw87xxx_set_profile(AW_DEV_0, aw_profile[0]);
+    else
+        aw87xxx_set_profile(AW_DEV_0, aw_profile[1]);
+    return HOOK_OK;
+}
+
+static void hook_gpio_pulse_control(unsigned int gpio, unsigned int mode)
+{
+    int i = 1;
+}
@@ -161,9 +187,14 @@ static int hook_general_spk(int id, int on)
    return HOOK_OK;
}
#endif

static struct sprd_asoc_ext_hook_map ext_hook_arr[] = {
#ifdef CONFIG_SND_SOC_AW87XXX:
+    {"aw87xx", hook_spk_aw87xxx, EN_LEVEL},
+
+    {"general_speaker", hook_general_spk, EN_LEVEL},
#endif
};
```

2.3 驱动移植有效性验证

以 aw87560 单 PA 注册举例：

1) I2C 通信正常：

```
[Awinic] aw87xxx_pa_init:driver version: Dv2.4.0.5
[Awinic] [0-0058]aw87xxx_malloc_init: struct aw87xxx devm_kzalloc and init down
[Awinic] [0-0058]aw87xxx_dtsi_parse: parse dev_index=[0]
[Awinic] [0-0058]aw87xxx_dtsi_parse: reset gpio[1160] parse succeed
[Awinic] [0-0058]aw87xxx_device_parse_port_id_dt: rx-port-id: 0xb030
[Awinic] [0-0058]aw87xxx_device_parse_topo_id_dt: rx-topo-id: 0x1000ff01
[Awinic] [0-0058]aw87xxx_dev_hw_pwr_ctrl: hw power on
[Awinic] [0-0058]aw87xxx_dev_get_chipid: read chipid[0x60] succeed
[Awinic] [0-0058]aw_dev_chip_init: product is pid_60 class
```

2) bin 文件加载正常：

```
[Awinic] [0-0058]aw_get_prof_count_v_0_0_0_1: get profile count=[3]
[Awinic] [0-0058]aw_get_vaild_prof_v_0_0_0_1: product=[aw87560]----profile=[Music]
[Awinic] [0-0058]aw_get_vaild_prof_v_0_0_0_1: product=[aw87560]----profile=[Receiver]
[Awinic] [0-0058]aw_set_prof_off_info_v_0_0_0_1: product=[aw87560]----profile=[Off]
[Awinic] [0-0058]aw_get_vaild_prof_v_0_0_0_1: get vaild profile succeed
[Awinic] [0-0058]aw_set_prof_name_list_v_0_0_0_1: index=[0], profile_name=[Music]
[Awinic] [0-0058]aw_set_prof_name_list_v_0_0_0_1: index=[1], profile_name=[Receiver]
[Awinic] [0-0058]aw_set_prof_name_list_v_0_0_0_1: index=[2], profile_name=[Off]
[Awinic] [0-0058]aw_parse_acf_v_0_0_0_1: acf parse success
[Awinic] [0-0058]aw87xxx_init_default_prof: init profile name [Off]
[Awinic] [0-0058]aw87xxx_fw_load: acf parse succeed
[Awinic] [0-0059]aw87xxx_fw_load: enter
```

3) Kcontrol 注册正常：

```
[Awinic] [0-0058]aw87xxx_kcontrol_dynamic_create: enter
[Awinic] [0-0058]aw87xxx_kcontrol_dynamic_create: add codec controls[aw87xxx_profile_switch_0,aw87xxx_vmax_get_0,aw87xxx_monitor_switch_0]
[Awinic] [0-0058]aw87xxx_public_kcontrol_create: enter
[Awinic] [0-0058]aw87xxx_public_kcontrol_create: add public codec controls[aw87xxx_spin_switch]
[Awinic] [0-0058]aw87xxx_profile_switch_get: current profile:[Off]
```

4) 切换场景成功（以设备节点方式为例）：

```
kona:/sys/bus/i2c/drivers/aw87xxx_pa/0-0058 # cat profile
Music
Receiver
>Off
kona:/sys/bus/i2c/drivers/aw87xxx_pa/0-0058 # echo Music > profile
[Awinic] [0-0058]aw87xxx_attr_get_profile: current profile:[Off]
[Awinic] [0-0058]aw87xxx_attr_set_profile: set profile [Music]
[Awinic] [0-0058]aw87xxx_update_profile: load profile[Music] enter
[Awinic] [0-0058]aw87xxx_monitor_stop: enter
[Awinic] [0-0058]aw87xxx_power_on: enter
[Awinic] [0-0058]aw87xxx_acf_get_prof_desc_form_name: enter
[Awinic] [0-0058]aw87xxx_acf_get_prof_desc_form_name: get prof desc down
[Awinic] [0-0058]aw87xxx_power_on: get profile[Music] data len [84]
```

5) 播放音乐正常：

```
[Awinic] aw87xxx_set_profile_by_id:aw87xxx, dev_index[0] set profile[Music] by id[1]
[Awinic] [0-0058]aw87xxx_set_profile: set dev_index = 0, profile = Music
[Awinic] [0-0058]aw87xxx_update_profile: load profile[Music] enter
[Awinic] [0-0058]aw87xxx_monitor_stop: enter
[Awinic] [0-0058]aw87xxx_power_on: enter
[Awinic] [0-0058]aw87xxx_acf_get_prof_desc_form_name: enter
[Awinic] [0-0058]aw87xxx_acf_get_prof_desc_form_name: get prof desc down
[Awinic] [0-0058]aw87xxx_power_on: get profile[Music] data len [84]
[Awinic] [0-0058]aw87xxx_dev_hw_pwr_ctrl: hw power on
[Awinic] [0-0058]aw87xxx_dev_reg_update: reg=0x76, val = 0x92
[Awinic] [0-0058]aw87xxx_dev_reg_update: reg=0x7d, val = 0x7a
[Awinic] [0-0058]aw87xxx_dev_reg_update: reg=0x7c, val = 0xc1
```

3. 调试接口

3.1 设备节点

AW87XXX Driver 会创建不同设备节点，路径是 `sys/bus/i2c/driver/aw87xxx_pa/*-00xx`，其中*为 i2c bus number，xx 为 i2c address。

reg

节点名字	reg
功能描述	用于读写 aw87xxx 的所有寄存器
使用方法	读寄存器值: <code>cat reg</code> 写寄存器值: <code>echo reg_addr reg_data > reg</code> (16 进制操作)
参考例程	<code>cat reg</code> (获取所有可读寄存器上的值) <code>echo 0x01 0x07 > reg</code> (向 0x01 寄存器写入 0x07 的值)

profile

节点名字	profile
功能描述	用于 aw87xxx 注册的 PA 切换场景
使用方法	查看当前场景和可切换的场景: <code>cat profile</code> 设置场景: <code>echo 场景名 > profile</code>
参考例程	<code>cat profile</code> (查看当前场景和可切换的场景) <code>echo "Music" > profile</code> (加载 Music 场景) <code>echo "Off" > profile</code> (加载 Off 场景)

hwen

节点名字	hwen
功能描述	用于控制 aw87xxx 的硬件使能

使用方法	获取 aw87xxx 硬件状态: cat hwen aw87xxx 硬件使能: echo 1 > hwen aw87xxx 硬件关闭: echo 0 > hwen
------	---

esd_enable

节点名字	esd_enable
功能描述	用于控制 esd 检测使能
使用方法	读: cat esd_enable 写: echo is_enable > esd_enable (字符串)
参考例程	cat esd_enable (获取 esd 功能是否使能) echo true > esd_enable (打开 esd 检测功能) echo false > esd_enable (关闭 esd 检测功能)

4. 附录

4.1 常见问题

4.1.1 bin 文件加载失败

1) 原因:

aw87xxx driver 需要 load 相关的配置文件, 但部分平台由于 selinux 权限问题导致 load bin 文件失败。

2) 解决方案:

在对应的 kernel.te 文件中添加缺少的权限。格式如下:

```
allow kernel vendor_file:file {getattr read}
```